JINK CSP-S 提高级模拟赛（一）题解

总体定位

本套题总分 400 分，建议比赛时间 4 小时，整体略低于近年正式 CSP-S 难度。

题目	核心算法	满分复杂度	预期定位
山脊维护	贪心公式、局部维护	$O(n+q)$	T1，提高组基础思维
气候分区	单调队列、双指针、前缀和 DP	$O(n)$	T2，提高组基础综合
观测窗口	权值转化、降维、单调队列	$O(\min(n,m)^2\max(n,m))$	T3，提高组基础综合
编号区间	LCA、树直径合并、线段树	$O((n+q)\log n)$	T4，提高组中档

第一题　山脊维护

令 $h_0=0$。对于一个固定的目标序列，答案为

$$F(h)=\sum_{i=1}^{n}\max(0,h_i-h_{i-1}).$$

公式证明

固定位置 i。如果 $h_i>h_{i-1}$，那么最终覆盖位置 i 的施工次数比覆盖位置 $i-1$ 的次数多 h_i-h_{i-1}。因此至少有 h_i-h_{i-1} 次施工必须从位置 i 开始。把所有正差相加，得到答案下界。

另一方面，从左向右扫描。当 $h_i>h_{i-1}$ 时，从位置 i 新开始 h_i-h_{i-1} 个区间；当高度下降时，结束相应数量的区间。这样恰好可以构造出 $F(h)$ 次施工，因此下界能够达到。

一次修改 $h_x\leftarrow y$ 只会改变

$$\max(0,h_x-h_{x-1})$$

以及（当 $x<n$ 时）

$$\max(0,h_{x+1}-h_x).$$

修改前从答案中减去这两项，修改后再加回即可。

时间复杂度 $O(n+q)$，空间复杂度 $O(n)$。注意答案需要使用 64 位整数。

第二题　气候分区

定义 dp_i 为把前 i 个位置合法划分的方案数，$dp_0=1$。

如果最后一段为 $[j+1,i]$，则有

$$dp_i=\sum dp_j,$$

其中 $i-R\le j\le i-L$，并且区间 $[j+1,i]$ 的极差不超过 D。

固定右端点

固定右端点 i。设 p_i 是最小的、使 $[p_i,i]$ 极差不超过 D 的左端点。 由于删除区间左端的元素不会让极差变大，所以所有满足极差限制的左端点恰好构成 $[p_i,i]$。于是合法的 j 构成一个连续区间：

$$\max(i-R,p_i-1)\le j\le i-L.$$

用两个单调队列分别维护当前窗口的最大值和最小值，即可随着 i 递增，以双指针方式求出 所有 p_i。再维护

$$pre_i=\sum_{j=0}^{i}dp_j,$$

每次转移就是一次前缀和区间求和。

时间复杂度 $O(n)$，空间复杂度 $O(n)$。

第三题　观测窗口

把每个观测点的维护成本直接计入它的权值，令

$$b_{i,j} = a_{i,j} - c.$$

于是任意窗口的净收益，恰好等于矩阵 b 中对应矩形的元素和，但矩形的行数必须属于 $[A, B]$，列数必须属于 $[L, R]$。

不妨先转置矩阵，使 $n \le m$。枚举窗口的上边界 `top`，并令一维数组 `sum[j]` 初始为 0。随后从 `top` 开始依次枚举下边界 `bottom`，把这一行的每个元素加入 `sum`：

$$sum_j = \sum_{i=top}^{bottom} b_{i,j}.$$

只有当 `bottom-top+1` 属于 $[A, B]$ 时，当前上下边界才可能构成合法窗口。设 `sum` 的一维前缀和为

$$pre_j = \sum_{t=1}^{j} sum_t.$$

若右端点为 r，合法左端点之前的位置 $p = l - 1$ 必须满足

$$r - R \le p \le r - L.$$

窗口的元素和为 $pre_r - pre_p$。因此固定 r 时，需要求上述区间内最小的 pre_p。随着 r 递增，候选下标的范围整体向右移动，可以用一个维护前缀和单调递增的双端队列：

- 把新下标 $r - L$ 加入队尾，并弹出队尾所有前缀和不小于它的下标；
- 从队首弹出所有小于 $r - R$ 的过期下标；
- 用 $pre_r - pre_{queue.front}$ 更新答案。

每个下标至多进入和离开队列一次，因此一组上下边界只需 $O(m)$ 时间。全负数据也无需特判，因为宽度下界 $L \ge 1$，队列中的候选始终对应非空区间。

枚举上边界后，下边界只需扩展到距离不超过 B 的位置；每组合法上下边界用 $O(m)$ 时间处理。转置矩阵时还要同时交换行、列长度限制。总时间复杂度为 $O(\min(n, m)^2 \max(n, m))$，空间复杂度为 $O(nm)$。答案需要使用 64 位整数。

第四题　编号区间

先用欧拉序和 ST 表预处理 LCA，使每次求两点距离只需 $O(1)$ 时间。

DFS 时，进入结点和从儿子返回时都记录当前结点，得到长度为 $2n - 1$ 的欧拉序。记 pos_x 为结点 x 第一次出现的位置。两个结点的 LCA，就是它们第一次出现的位置之间，深度最小的结点。用 ST 表维护区间内深度最小的结点，预处理 $O(n \log n)$，每次查询 $O(1)$。

两点距离为

$$dis(x, y) = dep_x + dep_y - 2dep_{lca(x,y)}.$$

对于一个非空点集 S，只保存它的一组直径端点 (x, y)。关键结论是：若点集 A, B 的直径端点分别为 (a_1, a_2) 和 (b_1, b_2)，那么 $A \cup B$ 的一组直径端点一定可以从下面六对中选出：

$$(a_1, a_2), (b_1, b_2), (a_1, b_1), (a_1, b_2), (a_2, b_1), (a_2, b_2).$$

这是树度量的经典性质：对于任意结点 z，点集 A 中距离 z 最远的结点一定可以在 A 的某一组直径端点中找到。因此跨越两个点集的最远点对，只需枚举两边的直径端点。

按结点编号建立线段树。叶结点 i 保存 (i, i)；内部结点使用上述规则合并左右儿子。询问编号区间 $[l, r]$ 时合并线段树覆盖该区间的结点，最后输出所得直径长度。

每次合并只需计算常数次距离，用上面的 ST 表可以在 $O(1)$ 时间内完成。 因此建线段树 $O(n)$，每次询问 $O(\log n)$。加上 LCA 预处理，总时间复杂度 $O((n+q)\log n)$，空间复杂度 $O(n\log n)$。

难度与预期得分说明

T1 的满分做法只需要发现固定序列答案的相邻差公式；T2 的 $O(n^2)$ 动态规划可以获得 前 20 分，在此基础上分别加入前缀和与双指针即可逐步接近满分；T3 可以用二维前缀和 枚举四条边界获得前 20 分，一维特殊情况可用单调队列，满分做法需要把二维问题降维后 再次使用单调队列；T4 的暴力距离和链结构均能获得部分分，满分要求识别树直径的可合并信息。

附录：标准程序

第 1 题　ridge 标准程序

```
#include<bits/stdc++.h>
#define int long long
#define endl "\n"
using namespace std;
const int N=1e6+10;
int n,q,a[N],ans;
int cal(int x,int y){return max(0ll,y-x);}
signed main()
{
    freopen("ridge.in","r",stdin);freopen("ridge.out","w",stdout);
    ios::sync_with_stdio(false);cin.tie(nullptr);
    cin>>n>>q;
    for(int i=1;i<=n;i++)cin>>a[i],ans+=cal(a[i-1],a[i]);
    while(q--)
    {
        int x,y;cin>>x>>y;
        ans-=cal(a[x-1],a[x]);
        if(x<n)ans-=cal(a[x],a[x+1]);
        a[x]=y;ans+=cal(a[x-1],a[x]);
        if(x<n)ans+=cal(a[x],a[x+1]);
        cout<<ans<<endl;
    }
    return 0;
}
```

第 2 题　climate 标准程序

```
#include<bits/stdc++.h>
#define int long long
#define endl "\n"
using namespace std;
const int N=1e6+10,P=1e9+7;
int n,L,R,D,a[N],f[N],sum[N];
deque<int> q1,q2;
signed main()
{
    freopen("climate.in","r",stdin);freopen("climate.out","w",stdout);
    ios::sync_with_stdio(false);cin.tie(nullptr);
    cin>>n>>L>>R>>D;for(int i=1;i<=n;i++)cin>>a[i];f[0]=sum[0]=1;int l=1;
    for(int i=1;i<=n;i++)
    {
        while(q1.size()&&a[q1.back()]<=a[i])q1.pop_back();
        q1.push_back(i);
        while(q2.size()&&a[q2.back()]>=a[i])q2.pop_back();
        q2.push_back(i);
        while(a[q1.front()]-a[q2.front()]>D)
        {
            if(q1.front()==l)q1.pop_front();
            if(q2.front()==l)q2.pop_front();
            l++;
        }
        int x=max(l-1,i-R),y=i-L;
        if(x<=y)f[i]=(sum[y]-(x?sum[x-1]:0)+P)%P;
        sum[i]=(sum[i-1]+f[i])%P;
    }
    cout<<f[n]<<endl;return 0;
```

```
}
```

第 3 题　window 标准程序

```
#include<bits/stdc++.h>
#define int long long
#define endl "\n"
using namespace std;
const int N=510,INF=1e18;
int n,m,h1,h2,w1,w2,c,a[N][N],sum[N],pre[N],q[N],ans=-INF;
signed main()
{
    freopen("window.in","r",stdin);
    freopen("window.out","w",stdout);
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin>>n>>m>>h1>>h2>>w1>>w2>>c;
    for(int i=1;i<=n;i++)
        for(int j=1;j<=m;j++)
        {
            int x;cin>>x;
            if(n<=m)a[i][j]=x-c;
            else a[j][i]=x-c;
        }
    if(n>m)swap(n,m),swap(h1,w1),swap(h2,w2);
    for(int l=1;l<=n;l++)
    {
        for(int j=1;j<=m;j++)sum[j]=0;
        for(int r=l;r<=n&&r-l+1<=h2;r++)
        {
            for(int j=1;j<=m;j++)sum[j]+=a[r][j];
            if(r-l+1<h1)continue;
            pre[0]=0;
            for(int j=1;j<=m;j++)pre[j]=pre[j-1]+sum[j];
            int L=1,R=0;
            for(int j=w1;j<=m;j++)
            {
                int x=j-w1;
                while(L<=R&&pre[q[R]]>=pre[x])R--;
                q[++R]=x;
                while(q[L]<j-w2)L++;
                ans=max(ans,pre[j]-pre[q[L]]);
            }
        }
    }
    cout<<ans<<endl;
    return 0;
}
```

第 4 题　interval 标准程序

```
#include<bits/stdc++.h>
#define int long long
#define endl "\n"
using namespace std;
const int N=2e5+10;
int n,q,tot,cnt,head[N],dep[N],pos[N],lg[N*2];
signed f[N*2][20];
struct edge
{
    int to,next;
}e[N*2];
struct node
{
    int x,y;
}t[N*4];
void add(int x,int y)
{
    e[++tot]={y,head[x]};
    head[x]=tot;
}
int calc(int x,int y)
{
    return dep[x]<dep[y]?x:y;
}
void dfs(int x,int fa)
{
```

```
    dep[x]=dep[fa]+1;
    pos[x]=++cnt;
    f[cnt][0]=x;
    for(int i=head[x];i;i=e[i].next)
    {
        int y=e[i].to;
        if(y==fa)continue;
        dfs(y,x);
        f[++cnt][0]=x;
    }
}
int lca(int x,int y)
{
    int l=pos[x],r=pos[y];
    if(l>r)swap(l,r);
    int k=lg[r-l+1];
    return calc(f[l][k],f[r-(1<<k)+1][k]);
}
int dis(int x,int y)
{
    if(!x||!y)return -1;
    return dep[x]+dep[y]-2*dep[lca(x,y)];
}
void upd(node &a,int &mx,int x,int y)
{
    int d=dis(x,y);
    if(d>mx)mx=d,a={x,y};
}
node merge(node a,node b)
{
    if(!a.x)return b;
    if(!b.x)return a;
    int x=a.x,y=a.y,mx=dis(x,y);
    upd(a,mx,b.x,b.y);upd(a,mx,x,b.x);upd(a,mx,x,b.y);
    upd(a,mx,y,b.x);upd(a,mx,y,b.y);
    return a;
}
void build(int p,int l,int r)
{
    if(l==r){t[p]={l,l};return;}
    int mid=(l+r)>>1;
    build(p<<1,l,mid);build(p<<1|1,mid+1,r);
    t[p]=merge(t[p<<1],t[p<<1|1]);
}
node query(int p,int l,int r,int L,int R)
{
    if(L<=l&&r<=R)return t[p];
    int mid=(l+r)>>1;
    node res={0,0};
    if(L<=mid)res=merge(res,query(p<<1,l,mid,L,R));
    if(R>mid)res=merge(res,query(p<<1|1,mid+1,r,L,R));
    return res;
}
signed main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cin>>n>>q;
    for(int i=1,x,y;i<n;i++)cin>>x>>y,add(x,y),add(y,x);
    dfs(1,0);
    for(int i=2;i<=cnt;i++)lg[i]=lg[i/2]+1;
    for(int j=1;(1<<j)<=cnt;j++)
        for(int i=1;i+(1<<j)-1<=cnt;i++)
            f[i][j]=calc(f[i][j-1],f[i+(1<<(j-1))][j-1]);
    build(1,1,n);
    while(q--)
    {
        int l,r;cin>>l>>r;
        node res=query(1,1,n,l,r);
        cout<<dis(res.x,res.y)<<endl;
    }
    return 0;
}
```